

Multi-Agent AI with AutoGen (AG2) + Google Gemini

Multi-Agent AI with AutoGen (AG2) + Google Gemini

Classroom Lab - Case Study: Should FinTrack launch an AI-powered receipt scanner?

Duration: ~45 minutes

Framework: AG2 (the actively-maintained fork of AutoGen - the two names are used interchangeably below)

Model: Google Gemini () - no OpenAI key is used anywhere in this notebook

What you will build

A team of 5 AI agents - Product Manager, Market Analyst, Technical Analyst, Security Analyst, Finance Analyst - that talk to each other in a shared conversation to decide whether a fictional company, FinTrack, should launch a new receipt-scanning feature.

Along the way you'll see every core multi-agent building block, each in its own tiny section:

Teaching tip: Run the cells top to bottom, in order. Later sections reuse agents and the object created in Section 1.

Section 1 - Setup *(~5 min)*

We install one package family: (PyPI package name), which is imported in Python as . and are the same package - think of as the modern name for the AutoGen project.

Important - package version: As of this writing, the project has two very different generations of API on PyPI:

- Classic API (versions) - the multi-agent building blocks this lab teaches: , , , tool-calling decorators, code executors.
- New API (versions) - a ground-up rewrite (, ,) with a completely different programming model, not what this lab needs.

We pin the version to (classic API) so the notebook stays reliable and consistent - mixing classic and new-generation APIs in one notebook is a common source of broken imports.

Code

```
# Install AG2 (imported as `autogen`) with the Gemini extras, pinned to the classic 0.x API.
# The [gemini] extra pulls in Google's google-genai / google-auth client libraries that AG2
# needs to talk to the Gemini API.
!pip install -q "ag2[gemini]==0.14.0"
```

Code

```
# --- Python version check -----
# ag2==0.14.0 requires Python 3.10 or newer. Colab's default runtime satisfies this, but we
# check explicitly and fail loudly (instead of silently continuing) if it doesn't.
import sys
MIN_PYTHON = (3, 10)
if sys.version_info < MIN_PYTHON:
    raise RuntimeError(
        f"This notebook requires Python {MIN_PYTHON[0]}.{MIN_PYTHON[1]}+ but the current "
        f"runtime is Python {sys.version_info.major}.{sys.version_info.minor}. "
        "In Colab: Runtime > Change runtime type, or use a newer Colab runtime image."
    )
print(f"Python version OK: {sys.version.split()[0]}")
```

Code

```

# --- Import verification -----
# We import every class this notebook will use, right now, so that any incompatible install
# fails immediately in Section 1 -- not halfway through the case study in Section 8.
import autogen # `autogen` is the import name for the `ag2` package we just installed
from autogen import (
    AssistantAgent,      # an LLM-backed agent that generates replies
    UserProxyAgent,      # an agent that represents "the code/human side" (can execute
                        code/tools)
    ConversableAgent,    # the base class both of the above inherit from
    GroupChat,           # holds a shared conversation between multiple agents
    GroupChatManager,    # orchestrates whose turn it is to speak in a GroupChat
    LLMConfig,           # wraps model + API settings that agents use to call the LLM
)
from autogen.coding import LocalCommandLineCodeExecutor # runs LLM-generated Python code
locally
from autogen.code_utils import content_str              # safely turns a message's content
into a string
print("autogen (ag2) version:", autogen.__version__)
print("All required classes imported successfully.")

```

Code

```
# --- Gemini API key -----
# getpass hides the key as you type it, so it never appears in the notebook output or history.
import getpass
GEMINI_API_KEY = getpass.getpass("Enter your Gemini API key: ")
# We ONLY use Google Gemini in this lab. gemini-3.5-flash is a fast, low-cost model that is a
# good fit for a classroom demo with many short agent turns.
GEMINI_MODEL = "gemini-3.5-flash"
# LLMConfig is the object every agent below will be given. "api_type": "google" tells AG2 to
# route requests through its Gemini client instead of OpenAI/Anthropic/etc.
llm_config = LLMConfig({
    "model": GEMINI_MODEL,
    "api_type": "google",
    "api_key": GEMINI_API_KEY,
})
print("LLM config ready for model:", GEMINI_MODEL)
```

Takeaway: every agent in this notebook is just a Python object that is handed the same
- the agents differ only in their (their personality and job).

Section 2 - Basic Agent *(~2 min)*

The smallest possible multi-agent building block: one AI agent answers one question.

* is the brain - it uses the LLM (Gemini) to generate replies, guided by its

.

* here plays the human asking the question. We set

so it doesn't pause to ask a real person for input - it just sends our message and stops.

* is how one agent starts a conversation with another. means

send one message, get one reply, then stop - perfect for a single Q&A.

Code

```
# The "brain": answers questions using Gemini, guided by its system_message.
basic_assistant = AssistantAgent(
    name="Basic_Agent",
    system_message="You are a helpful assistant. Explain AI concepts simply, in 3-4
    sentences.",
    llm_config=llm_config,
)
# The "asker": sends one message and does not auto-reply (no LLM needed for this agent).
asker = UserProxyAgent(
    name="Asker",
    human_input_mode="NEVER",          # never pause for real human input
    max_consecutive_auto_reply=0,      # after receiving a reply, do not send anything back
    code_execution_config=False,       # this agent does not run any code
)
# Kick off a one-turn conversation: Asker -> Basic_Agent -> (one reply) -> stop.
basic_chat_result = asker.initiate_chat(
    basic_assistant,
    message="What is an AI agent?",
    max_turns=1,
)
```

Code

```
# The full back-and-forth is stored as a list of {"role": ..., "content": ...} dicts.
print("Basic_Agent's answer:\n")
print(basic_chat_result.chat_history[-1]["content"])
```

Takeaway: an agent is just an LLM call wrapped with a name, a system message, and a way to send/receive messages.

Section 3 - Two-Agent Conversation *(~5 min)*

Now two AI-backed agents talk to each other:

- Product Expert proposes a short idea for the receipt scanner.
- Reviewer critiques it.

This is the first time you'll see agents genuinely communicating - the Reviewer's reply is generated by reading the Product Expert's message, not by us writing it.

Code

```
product_expert = AssistantAgent(
    name="Product_Expert",
    system_message=(
        "You are a product expert at FinTrack. Propose a short (2-3 sentence) idea for how "
        "the AI receipt scanner feature should work for customers."
    ),
    llm_config=llm_config,
)
reviewer = AssistantAgent(
    name="Reviewer",
    system_message=(
        "You are a critical product reviewer. When given a product idea, point out ONE clear "
        "weakness or risk in 2-3 sentences. Be specific, not generic."
    ),
    llm_config=llm_config,
)
# Product_Expert INITIATES the chat with Reviewer, so Product_Expert speaks first with our
# message, and Reviewer replies once (max_turns=1 = one full back-and-forth exchange).
two_agent_result = product_expert.initiate_chat(
    reviewer,
    message="Propose a short idea for FinTrack's AI receipt scanner.",
    max_turns=1,
)
```

Code

```
# chat_history shows both sides of the conversation in order -- this IS agent-to-agent
# communication: no human wrote the Reviewer's critique, Gemini generated it by reading
# Product_Expert's message.
for turn in two_agent_result.chat_history:
    speaker = turn.get("name") or turn.get("role", "unknown") # some messages have no "name"
    key
    print(f"--- {speaker} ---")
    print(turn["content"])
    print()
```

Takeaway: between two s is a real conversation - each agent reads and responds to what the other actually said.

Section 4 - Sequential Chain *(~4 min)*

A very common pattern: the output of one agent becomes the input to the next.

produces 3 short market-research bullet points. We then take that text and hand it, as-is, into the message we send to .

Code

```

research_agent = AssistantAgent(
name="Research_Agent",
system_message="You research markets. Answer with exactly 3 short bullet points, nothing
else.",
llm_config=llm_config,
)
summary_agent = AssistantAgent(
name="Summary_Agent",
system_message="You summarize research into ONE concise sentence.",
llm_config=llm_config,
)
# A plain, non-LLM "runner" agent we reuse just to send single messages and collect replies.
runner = UserProxyAgent(
name="Runner",
human_input_mode="NEVER",
max_consecutive_auto_reply=0,
code_execution_config=False,
)
# Step 1: get research points out of Research_Agent.
research_result = runner.initiate_chat(
research_agent,
message="List 3 short bullet points about the market potential for an AI receipt scanner.",
max_turns=1,
)
research_points = research_result.chat_history[-1]["content"] # the agent's reply text
print("Research_Agent's output:\n")
print(research_points)

```

Code

```
# Step 2: feed Research_Agent's OUTPUT directly into Summary_Agent's INPUT message.
summary_result = runner.initiate_chat(
    summary_agent,
    message=f"Summarize the following market research into ONE sentence:\n\n{research_points}",
    max_turns=1,
)
summary_text = summary_result.chat_history[-1]["content"]
print("Summary_Agent's output:\n")
print(summary_text)
```

Takeaway: chaining agents is just: run agent A, grab its text reply, put that text into agent Bs message.

Section 5 - Reflection ***(~4 min)***

Reflection = an agents own output is critiqued, and then the agent improves it.

? ? (revises based on the critique). This is how multi-agent systems self-correct without a human stepping in each time.

Code

```

writer = AssistantAgent(
    name="Writer",
    system_message="You write short, confident product launch recommendations (1 sentence).",
    llm_config=llm_config,
)
critic = AssistantAgent(
    name="Critic",
    system_message=(
        "You review launch recommendations. In 1-2 sentences, list what important risks or "
        "considerations the recommendation is MISSING (e.g. privacy, accuracy, cost).",
    ),
    llm_config=llm_config,
)
# --- Draft ---
draft_result = runner.initiate_chat(
    writer,
    message="Write a 1-sentence launch recommendation for FinTrack's AI receipt scanner.",
    max_turns=1,
)
draft_text = draft_result.chat_history[-1]["content"]
print("DRAFT:\n", draft_text, "\n")

```

Code

```

# --- Critique ---
critique_result = runner.initiate_chat(
    critic,
    message=f"Critique this recommendation:\n\n{draft_text}",
    max_turns=1,
)

```

Code

```
# --- Revision: Writer sees its own draft AND the critique, and improves it ---
revision_result = runner.initiate_chat(
    writer,
    message=(
        f"Your original recommendation was:\n{draft_text}\n\n"
        f"A reviewer raised this feedback:\n{critique_text}\n\n"
        "Write a revised 1-2 sentence recommendation that addresses the feedback."
    ),
    max_turns=1,
)
revised_text = revision_result.chat_history[-1]["content"]
print("REVISED:\n", revised_text)
```

Takeaway: reflection is a 3-step loop draft ? critique ? revise and it visibly produces a better, more cautious answer than the first draft.

Section 6 - Tool / Function Calling *(~3 min)*

So far, agents have only generated text. Now an agent will decide, on its own, that it needs to call a real Python function to answer accurately (LLMs are bad at precise arithmetic).

The flow: Agent decides a tool is needed ? Python function runs ? result returns to the agent ? agent answers using that result. We use AG2s real tool-calling mechanism

(/) - Gemini genuinely emits a tool call, AG2 executes

the Python function, and the result is sent back to Gemini automatically.

Code

```

tool_assistant = AssistantAgent(
    name="Finance_Bot",
    system_message=(
        "You are a finance assistant. When asked about ROI, ALWAYS use the calculate_roi tool "
        "instead of calculating by hand. Report the final ROI percentage clearly."
    ),
    llm_config=llm_config,
)
# This agent EXECUTES the tool once Finance_Bot decides to call it.
tool_executor = UserProxyAgent(
    name="Tool_Executor",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=3,    # allow a couple of automatic back-and-forth tool round
    trips
    code_execution_config=False,      # no free-form code execution here, only the registered
    tool
)
# @tool_executor.register_for_execution()    -> "when this tool is called, Tool_Executor runs
it"
# @tool_assistant.register_for_llm(...)      -> "tell Finance_Bot's LLM this tool exists"
@tool_executor.register_for_execution()
@tool_assistant.register_for_llm(description="Calculate ROI as a percentage from revenue and
cost.")
def calculate_roi(revenue: float, cost: float) -> str:
    roi_percent = ((revenue - cost) / cost) * 100
    return f"{roi_percent:.1f}%"
print("Tool 'calculate_roi' registered on Finance_Bot.")

```

Code

```
tool_result = tool_executor.initiate_chat(
    tool_assistant,
    message="If the receipt scanner is projected to bring in $150,000 revenue at a cost of $50,000, what is the ROI?",
    max_turns=3,
)
for turn in tool_result.chat_history:
    # Not every message has a "name" key -- a "Suggested tool call" message from Finance_Bot
    # only has role/content/tool_calls, so we fall back to the role instead.
    speaker = turn.get("name") or turn.get("role", "unknown")
    print(f"--- {speaker} ---")
    text = content_str(turn.get("content"))
    if not text and turn.get("tool_calls"):
        # This turn IS a tool-call request -- show what was requested, not raw content.
        for call in turn["tool_calls"]:
            fn = call.get("function", {})
            text = f'[tool call] {fn.get("name")}({fn.get("arguments")})'
        elif not text and turn.get("tool_responses"):
            # This turn IS a tool result coming back.
            text = "\n".join(str(r.get("content")) for r in turn["tool_responses"])
    print(text)
    print()
```

Takeaway: tool calling lets an LLM offload precise work (math, lookups, database queries) to real code instead of guessing.

Section 7 - Code Execution *(~2 min)*

Different from tool calling: here the agent writes a snippet of Python code itself, and actually runs it (safely, as a local Python subprocess - no shell)

commands, no Docker required).

Tool calling = call one predefined, safe function.

Code execution = the agent writes arbitrary (but sandboxed) code to solve a task.

Code

```
# The executor is the thing that actually runs any Python code block the agent writes.
# work_dir is just a folder where the generated code files are temporarily saved.
code_executor = LocalCommandLineCodeExecutor(timeout=30, work_dir="coding")
coder_assistant = AssistantAgent(
    name="Coder",
    system_message=(
        "You are a Python coding assistant. Write a short Python code block to solve the "
        "user's request. After the code has been executed and you see the result, reply with "
        "exactly the word DONE (do not say DONE in the same message as code).",
    ),
    llm_config=llm_config,
)
code_runner = UserProxyAgent(
    name="Code_Runner",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=3,
    code_execution_config={"executor": code_executor}, # <-- this is what makes execution
    is_termination_msg=lambda msg: "DONE" in content_str(msg.get("content")),
)
code_exec_result = code_runner.initiate_chat(
    coder_assistant,
    message="Calculate 120000 + 80000 + 95000 using Python and print the result.",
)
```

Takeaway: code execution is how agents solve problems that are hard to describe as a single tool they write and run real code to get the answer, and you see the actual output, not a guess.

Section 8 - Main Case Study: The FinTrack GroupChat *(~12 min)*

Everything so far has been 1-on-1. Now: five agents share ONE conversation and respond to each other, the way a real product team would in a meeting.

Each agent below is defined with:

1. Its role in the business decision,
2. Its responsibilities (what it must talk about),
3. Its personality/stance (what argument it should push).

We use + - the shared meeting room and its facilitator.

makes the turn order predictable (each agent speaks in a fixed rotation), which is the simplest and most reliable choice for a live classroom demo - no risk of the LLM picking a confusing or repetitive next speaker.

Code

```

product_manager = AssistantAgent(
    name="Product_Manager",
    system_message=(
        "You are the Product Manager at FinTrack. You OWN the launch decision. "
        "You are pragmatic, concise, and decision-oriented. You keep the discussion focused, "
        "ask the other analysts for evidence, point out disagreements between them, and near "
        "the end you summarize the discussion and give a final recommendation. "
        "Do NOT simply agree with everyone -- push back when arguments are weak or incomplete. "
        "Keep every message to 2-4 sentences."
    ),
    llm_config=llm_config,
)
market_analyst = AssistantAgent(
    name="Market_Analyst",
    system_message=(
        "You are the Market Analyst. You argue that the AI receipt scanner could reduce manual "
        "receipt entry and improve customer experience, but you are honest that adoption "
        "depends "
        "heavily on extraction accuracy and customer trust. Respond directly to points made by "
        "other agents. Keep every message to 2-4 sentences."
    ),
    llm_config=llm_config,
)
technical_analyst = AssistantAgent(
    name="Technical_Analyst",
    system_message=(
        "You are the Technical Analyst. You explain that the feature is technically feasible "
        "using OCR and LLM-based extraction, but receipt formats vary significantly, so "
        "extracted "
        "data MUST be validated before being trusted. You flag integration risk with existing "
        "systems. Keep every message to 2-4 sentences."
    ),
    llm_config=llm_config,
)
security_analyst = AssistantAgent(
    name="Security_Analyst",
    system_message=(
        "You are the Security Analyst. Your job is to CHALLENGE the proposal from a privacy "
        "and "
        "security angle. Raise concerns about personal and financial information in receipts, "
        "image storage, access control, and data retention. Actively disagree with overly "
        "optimistic claims from other agents when they ignore privacy risk. "
        "Keep every message to 2-4 sentences."
    ),
    llm_config=llm_config,
)
finance_analyst = AssistantAgent(
    name="Finance_Analyst",
    system_message=(
        "You are the Finance Analyst. You evaluate commercial viability: implementation cost, "
        "ongoing AI/API/infrastructure cost, and expected ROI. You challenge the team to "
        "justify "
        "business value against operating cost, and suggest a pilot rather than a full launch "
        "when the numbers are uncertain. Keep every message to 2-4 sentences."
    ),
    llm_config=llm_config,
)
print("All 5 case-study agents created.")

```

Code

```
# The GroupChat is the shared "room" all 5 agents are in.
# - agents: everyone allowed to participate
# - max_round: hard cap on total turns, so the discussion can't run away (keeps API calls low)
# - speaker_selection_method="round_robin": agents speak in a fixed, repeating order --
#   simple and predictable for a live class
groupchat = GroupChat(
    agents=[product_manager, market_analyst, technical_analyst, security_analyst,
    finance_analyst],
    messages=[],
    max_round=11,
    speaker_selection_method="round_robin",
)
# The GroupChatManager is the "facilitator": it receives each agent's message and passes the
# floor to the next speaker according to the GroupChat's rules above.
manager = GroupChatManager(
    name="chat_manager",
    groupchat=groupchat,
    llm_config=llm_config,
)
print("GroupChat ready with", len(groupchat.agents), "agents, max_round =",
groupchat.max_round)
```

Code

```

CASE_STUDY_PROMPT = (
    "FinTrack is considering launching an AI-powered receipt scanner next quarter. The feature "
    "lets customers upload or photograph a receipt, and AI extracts the merchant, date, items, "
    "total, tax, and spending category. Product_Manager, please open the discussion: should "
    "FinTrack launch it? Discuss customer value, technical feasibility, security/privacy, and "
    "cost -- then summarize with a final recommendation."
)
# Product_Manager sends the opening prompt into the GroupChat via the manager.
# From here, GroupChatManager automatically rotates speakers (round-robin) for up to
# groupchat.max_round turns, with each agent reading the FULL shared history so far.
case_study_result = product_manager.initiate_chat(
    manager,
    message=CASE_STUDY_PROMPT,
)

```

Code

```

# Replay the whole team discussion in order -- this is the "visible disagreement and response"
# the case study asked for.
print("=" * 70)
print("FULL TEAM DISCUSSION")
print("=" * 70)
for turn in groupchat.messages:
    speaker = turn.get("name", "unknown")
    text = content_str(turn.get("content"))
    if text:
        print(f"\n[{speaker}]\n{text}")

```

Takeaway: a is a shared message history + a turn-taking rule every agent sees everything said so far, which is exactly why they can agree, disagree, and build on each others points.

Section 9 - Human-in-the-Loop *(~3 min)*

Even with 5 AI agents debating, a human makes the final call. This is a critical safety pattern: AI teams can prepare a recommendation, but a person decides whether to act on it.

We pull the Product Managers final message out of the GroupChat as the recommendation, show it, and then pause for a real decision from you (the student).

Code

```
# The Product Manager's LAST message in the group chat is treated as the team's
recommendation.
pm_messages = [t for t in groupchat.messages if t.get("name") == "Product_Manager"]
team_recommendation = content_str(pm_messages[-1]["content"]) if pm_messages else "(no
recommendation found)"
print("=" * 70)
print("TEAM RECOMMENDATION")
print("=" * 70)
print(team_recommendation)
```

Code

```
# input() pauses the notebook and waits for you to type a response -- this IS the human in the
loop.
decision = input("Do you approve this recommendation? (approve/reject): ").strip().lower()
if decision == "approve":
    print("Human approved. Proceed to launch planning.")
elif decision == "reject":
    print("Human rejected the recommendation. The team must reconsider.")
else:
    print("Unrecognized input. Please re-run this cell and type 'approve' or 'reject'.")
```

Takeaway: human-in-the-loop can be as simple as one call gating what happens next the AI team never gets to act unilaterally.

Section 10 - Mini Text2SQL (ReAct-style) *(~5 min)*

Last concept: an agent answering a data question by reasoning about which action to take, taking that action, observing the result, and then answering the core idea behind ReAct (Reason + Act) agents.

We reuse the exact tool-calling mechanism from Section 6: the agent doesn't run SQL directly it calls a registered tool, and AG2 executes it and returns real rows.

Code

```
import sqlite3
# An in-memory SQLite database -- no external database server, and it disappears when the
# notebook's Python process ends.
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute("CREATE TABLE sales (product TEXT, region TEXT, amount INTEGER)")
cursor.executemany(
    "INSERT INTO sales (product, region, amount) VALUES (?, ?, ?)",
    [
        ("Laptop", "North", 120000),
        ("Phone", "North", 80000),
        ("Laptop", "South", 95000),
        ("Phone", "South", 70000),
    ],
)
conn.commit()
print("In-memory 'sales' table created with 4 rows.")
```

Code

```
sql_assistant = AssistantAgent(
    name="Data_Analyst",
    system_message=(
        "You answer questions about a 'sales' SQLite table with columns: product, region, "
        "amount. The table ALREADY EXISTS -- do not try to create it. ALWAYS use the run_sql "
        "tool to query the data; never guess numbers. After you get the result, answer the "
        "user's question in one clear sentence."
    ),
    llm_config=llm_config,
)
sql_executor = UserProxyAgent(
    name="SQL_Executor",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=3,
    code_execution_config=False,
)
@sql_executor.register_for_execution()
@sql_assistant.register_for_llm(description="Run a read-only SQL SELECT query against the
'sales' table and return the rows.")
def run_sql(query: str) -> str:
    # THINK: the agent decided a query was needed and generated `query`.
    # ACT: we execute it here.
    cursor.execute(query)
    rows = cursor.fetchall()
    # OBSERVE: the rows are returned back to the agent as its next input.
    return str(rows)
print("Tool 'run_sql' registered on Data_Analyst.")
```

Code

```
sql_result = sql_executor.initiate_chat(
    sql_assistant,
    message="What is the total sales amount by region?",
    max_turns=3,
)
for turn in sql_result.chat_history:
    text = content_str(turn.get("content"))
    if text:
        speaker = turn.get("name") or turn.get("role", "unknown") # some messages have no
        "name" key
        print(f"--- {speaker} ---")
        print(text)
        print()
    # Expected underlying numbers, for reference: North = 200000, South = 165000
```

Takeaway: ReAct is just tool calling applied to reasoning the agent decides an action is needed (query the DB), takes it, observes the result, and only then answers.

Section 11 (Bonus) - Actually Scanning a Receipt with Gemini Vision *(~6 min, optional)*

Every section so far only ever talked about the receipt scanner feature - no image was ever uploaded or read. This bonus section is the one place in the notebook where that changes: we send a real receipt photo to Gemini and get real extracted fields back.

- * is a version of that understands tags inside a message and sends the image to the model alongside the text.
- * Gemini reads the pixels and returns the merchant, date, items, total, tax, and category as text.
- * This is the one part of the lab that is not simulated if the extraction is wrong or messy, that IS the real-world accuracy problem the Technical Analyst was warning about in Section 8.

Code

```

# We need Python's imaging library to draw a sample receipt, and AG2's multimodal agent class
# to send an image alongside text to Gemini.
from PIL import Image, ImageDraw, ImageFont
from IPython.display import display
from autogen.agentchat.contrib.multimodal_conversable_agent import MultimodalConversableAgent
RECEIPT_IMAGE_PATH = None
# --- Option A: upload your own receipt photo (only works inside Google Colab) ---
try:
    from google.colab import files
    print("Upload a receipt photo (.jpg/.png), or click Cancel to use the built-in sample
    instead.")
    uploaded = files.upload()
    if uploaded:
        RECEIPT_IMAGE_PATH = next(iter(uploaded.keys()))
        print("Using your uploaded file:", RECEIPT_IMAGE_PATH)
    except ImportError:
        pass # not running in Colab -- fall through to the generated sample below
# --- Option B: no upload happened, so generate a simple sample receipt to scan instead ---
if RECEIPT_IMAGE_PATH is None:
    W, H = 500, 700
    img = Image.new("RGB", (W, H), "white")
    d = ImageDraw.Draw(img)
    font_b = ImageFont.load_default(size=20)
    font = ImageFont.load_default(size=16)
    font_sm = ImageFont.load_default(size=13)
    y = 30
    d.text((150, y), "GREENLEAF CAFE", font=font_b, fill="black"); y += 30
    d.text((120, y), "482 Market Street, Kota", font=font_sm, fill="black"); y += 20
    d.text((150, y), "Tel: 555-0182", font=font_sm, fill="black"); y += 35
    d.line((30, y, W - 30, y), fill="black", width=1); y += 15
    d.text((30, y), "Date: 2026-08-14   Time: 10:42 AM", font=font_sm, fill="black"); y += 25
    d.line((30, y, W - 30, y), fill="black", width=1); y += 15
    for name, price in [("Cappuccino (L)", "180.00"), ("Blueberry Muffin", "120.00"),
    ("Veg Sandwich", "220.00"), ("Bottled Water", "40.00")]:
        d.text((30, y), name, font=font, fill="black")
        d.text((W - 30 - len(price) * 10, y), price, font=font, fill="black")
        y += 28
    y += 5
    d.line((30, y, W - 30, y), fill="black", width=1); y += 15
    d.text((30, y), "Subtotal", font=font, fill="black"); d.text((W - 100, y), "560.00",
    font=font, fill="black"); y += 24
    d.text((30, y), "Tax (5%)", font=font, fill="black"); d.text((W - 100, y), "28.00",
    font=font, fill="black"); y += 24
    d.line((30, y, W - 30, y), fill="black", width=1); y += 15
    d.text((30, y), "TOTAL", font=font_b, fill="black"); d.text((W - 120, y), "588.00",
    font=font_b, fill="black"); y += 40
    d.text((30, y), "Category: Food & Dining", font=font_sm, fill="black")
    RECEIPT_IMAGE_PATH = "sample_receipt.png"
    img.save(RECEIPT_IMAGE_PATH)
    print("No upload detected -- generated a sample receipt instead:", RECEIPT_IMAGE_PATH)
    # Show whichever image we ended up with, right in the notebook.
    display(Image.open(RECEIPT_IMAGE_PATH))

```

Code

```
# A vision-capable agent: same idea as AssistantAgent, but it also understands <img ...> tags.
receipt_vision_agent = MultimodalConversableAgent(
    name="Receipt_Vision_Agent",
    system_message=(
        "You extract structured data from receipt photos. Given a receipt image, respond with "
        "exactly these fields, one per line: Merchant, Date, Items (comma-separated), Total, "
        "Tax, "
        "Category. If a field is unclear or missing, write 'unclear' for that field -- never "
        "guess."
    ),
    llm_config=llm_config,
    max_consecutive_auto_reply=1,
)
uploader = UserProxyAgent(
    name="Uploader",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=0,
    code_execution_config=False,
)
# The <img PATH> tag is what tells AG2 to attach the actual image bytes to this message.
extraction_result = uploader.initiate_chat(
    receipt_vision_agent,
    message=f"Extract the fields from this receipt: <img {RECEIPT_IMAGE_PATH}>",
    max_turns=1,
)
print("EXTRACTED FIELDS:\n")
print(extraction_result.chat_history[-1]["content"])
```

Takeaway: this is the only real implementation of the receipt scanner in the whole notebook everything in Section 8 was agents debating whether to build this. Compare the extraction above against the actual receipt image: any mistakes you see are a live example of the formats vary, results must be validated risk the Technical Analyst raised in the case study.

Recap *(~2 min)*

The big idea: a multi-agent system is not magic its a handful of LLM-backed agents, each with a clear job description, sharing messages according to simple, explicit rules (one-on-one, sequential, or a shared GroupChat) with a human still holding final approval.